

TRAFFIC / PROTOCOL INVESTIGATION USING WIRESHARK

Network Traffic Analysis Project

Submitted by:

Nanditha

Student ID: 1AT24CS132

Department of Computer Science and Engineering

Course: Cybersecurity Fundamentals / Network Traffic Analysis

Date: August 2026

Tools Used:

Wireshark (Network Protocol Analyzer) · Windows OS with Wi-Fi Interface

Table of Contents

Abstract.....	3
Objective	3
Tools and Environment	4
Methodology	4
6. Detailed Traffic Investigation	6
Figure 1 - Main Wireshark Capture (ip.addr == 192.168.1.4).....	6
Figure 2 - mDNS Filtered Traffic and Packet Detail	7
Figure 3 - TCP Stream 0: Connection to 192.168.1.4:8009	8
Figure 4 - Follow TCP Stream: Encrypted Payload.....	8
Figure 5 - TLS 1.2 Packet Detail: Application Data.....	9
Figure 6 - mDNS and IP Filter: Google Cast Query	9
Figure 7 - mDNS SRV Record: Google Cast on Port 8009	10
Figure 8 - TLS Handshake Filter Returns No Results.....	10
Figure 9 - TCP Conversation Statistics	11
Figure 10 - Protocol Hierarchy Statistics	11
Figure 11 - DNS Response for microsoft.com Endpoint	12
Figure 12 - TCP Connection to 4.150.223.96:443.....	12
Figure 13 - Additional TCP Connection (Port 5228).....	13
7. Specific Investigation Findings	14
7.1 Local Device Identification	14
7.2 mDNS Investigation	14
7.3 Google Cast Service Investigation	14
7.4 TCP Port 8009 Investigation	15
7.5 TLS Investigation.....	15
7.6 TCP Conversation Statistics	15
7.7 Protocol Hierarchy.....	16
7.8 DNS Investigation	16
7.9 DNS → TCP/TLS Correlation	16
7.10 Additional TCP Connection - Port 5228.....	17
8. Protocol Explanation	18
9. Traffic Flow Diagram	19
10. Key Findings	20
11. Security Analysis	21
12. Limitations	22
13. Conclusion	23
14. Skills Learned	23
15. Future Improvements.....	24
Overall Investigation Summary.....	25

Abstract

This project report documents a beginner-level network traffic investigation performed using Wireshark, a widely used open-source packet analyzer. The investigation was conducted on a live Wi-Fi network, capturing real network traffic between local devices and the internet over a limited observation period.

The primary objectives were to identify active devices on the local network, understand what protocols they were using, discover what services they were running, and correlate low-level packet evidence with real-world network behaviors - all without making assumptions beyond what the captured packets directly supported.

During the investigation, two local devices were identified: 192.168.1.7 (the local machine from which the capture was performed) and 192.168.1.4, which was discovered to be a BPL TV / Android-related device. This device was found to be actively advertising a Google Cast service via the mDNS (Multicast DNS) protocol. A captured SRV (Service Record) confirmed that the Google Cast service was bound to TCP port 8009 on 192.168.1.4.

A TCP connection between 192.168.1.7 and 192.168.1.4:8009 was examined. TLS 1.2 encryption was found to protect the application data, making the payload unreadable through direct inspection. The protocol hierarchy revealed a diverse capture including IPv4, IPv6, TCP, UDP, TLS, mDNS, DNS, ARP, and QUIC traffic. Additionally, a DNS query for mobile.events.data.microsoft.com was captured, resolving to 4.150.223.96 through Azure infrastructure. Corresponding TCP/TLS traffic to that IP address was found, demonstrating a complete DNS-to-connection workflow. A second TCP connection from 172.217.70.188 on source port 5228 was also observed and analyzed in context.

This project demonstrated that packet capture and protocol analysis are powerful tools for understanding network behavior, identifying devices, and beginning to reason about security - even at a beginner level.

Objective

The primary purpose of this project was to perform a structured, evidence-based investigation of real network traffic using Wireshark. The investigation was approached as a practical exercise in network forensics and traffic analysis, with the following specific objectives:

- Capture real network traffic on a live Wi-Fi interface and observe what devices and protocols are present.
- Identify devices on the local network by analyzing their IP addresses and the traffic they generate.
- Understand the role of TCP/IP communication by examining packet-level details including source/destination addresses, ports, sequence numbers, and acknowledgment numbers.
- Investigate mDNS (Multicast DNS) traffic to understand how devices discover services on a local network without a central DNS server.
- Identify what services a local device is running based on the content of its mDNS advertisements.
- Understand TLS (Transport Layer Security) encryption - specifically, how it protects data and why it prevents direct payload inspection even when packets are captured.
- Correlate a DNS SRV record with an observed TCP port to establish the port-service relationship through evidence rather than assumption.
- Perform basic DNS investigation: capture a DNS query and response, identify the resolved IP address, and correlate that resolution with observed TCP/TLS traffic.
- Analyze conversation statistics and protocol hierarchy to understand the overall composition of a packet capture.
- Practice responsible, evidence-based interpretation - clearly distinguishing between what the capture proves and what it does not.

Tools and Environment

Primary Tool

Wireshark

Wireshark is a free and open-source network protocol analyzer. It captures packets traveling across a network interface and allows detailed inspection of every layer of the network stack - from the physical Ethernet frame through the application-layer protocol. Wireshark is one of the most widely used tools in network troubleshooting, security analysis, and protocol research.

Operating System and Platform

The capture was performed on a Windows machine. The capture interface shown in the Wireshark title bars indicates a Wi-Fi connection (the interface is labeled "Wi-Fi"). The device hostnames and MAC addresses visible in the packet details confirm a standard IEEE 802.11 wireless network environment.

Network Environment

The investigation was conducted on a local Wi-Fi network using the 192.168.1.0/24 subnet - a standard private home or office network range. Two primary local devices were involved: 192.168.1.7 (the capture machine) and 192.168.1.4 (a BPL TV / Android-related device). Internet-facing traffic was also captured, involving external IPs such as 4.150.223.96 (Microsoft Azure) and 172.217.70.188 (Google infrastructure).

Capture Interface

The Wi-Fi adapter was used as the capture interface, as confirmed by the Wireshark window title bars displaying "Wi-Fi" throughout the investigation screenshots.

Methodology

The investigation followed a logical, step-by-step process - beginning with a broad capture and progressively narrowing the focus to specific devices, services, ports, and protocols. Each step was evidence-driven: conclusions were drawn only from information visible in the captured packets.

Step 1 - Capture Network Traffic

Wireshark was launched on the Wi-Fi interface and allowed to capture live network traffic for a period of time. The resulting capture contained 132 total frames across multiple protocols, as confirmed by the protocol hierarchy statistics.

Step 2 - Identify Traffic Involving the Local Device

The Wireshark display filter `ip.addr == 192.168.1.4` was applied to isolate all traffic involving the device at 192.168.1.4. This immediately revealed a mix of mDNS broadcasts and TLS/TCP traffic, indicating that this device was both communicating with the local machine and actively advertising services on the network.

Step 3 - Investigate mDNS

The filter was changed to `mdns` to observe all Multicast DNS traffic in the capture. mDNS is used on local networks for zero-configuration service discovery - devices broadcast their available services without needing a central DNS server.

Step 4 - Identify the BPL TV / Android-Related Device

Within the mDNS traffic, the device at 192.168.1.4 was found to be advertising itself under the names "BPL TV._androidtvremote2._tcp.local" and related identifiers, clearly marking it as an Android TV device manufactured by BPL.

Step 5 - Investigate TCP Port 8009

The mDNS traffic referenced a service on port 8009. A filter was applied to isolate the TCP conversation between 192.168.1.7 and 192.168.1.4 on this port, revealing an active bidirectional connection using TLS 1.2 encryption.

Step 6 - Identify the Google Cast Service

Returning to the mDNS traffic, a specific query and response for `_googlecast._tcp.local` was identified, directly linking the Google Cast service to 192.168.1.4.

Step 7 - Correlate the Google Cast SRV Record with Port 8009

A DNS SRV record in the mDNS response explicitly stated: port 8009, target hostname 47339214-b58c-fc2d-51bb-b83b374d7659.local, and an A record resolving that hostname to 192.168.1.4. This provided direct evidence of the port-service relationship.

Step 8 - Investigate TLS Encryption

The Follow TCP Stream function was used on the 192.168.1.7 ↔ 192.168.1.4:8009 connection. The stream content was unreadable due to TLS encryption. A separate filter (`tls.handshake && ip.addr == 192.168.1.4 && tcp.port == 8009`) was applied to look for the TLS handshake, which returned no results - indicating the handshake was not present in this capture window.

Step 9 - Analyze TCP Conversation Statistics

The Statistics → Conversations panel was opened and the TCP tab was examined. A single TCP conversation was visible: 192.168.1.7 (port 57016) ↔ 192.168.1.4 (port 8009), involving 11 packets totaling 1 KB.

Step 10 - Analyze the Overall Protocol Hierarchy

Statistics → Protocol Hierarchy was opened to gain an overview of all protocols in the capture. This revealed 132 total frames with a rich mix of protocols across both IPv4 and IPv6, including TCP, UDP, TLS, mDNS, DNS, QUIC, ARP, and ICMPv6.

Step 11 - Investigate DNS Resolution

A DNS query for mobile.events.data.microsoft.com was located in the capture. The response contained CNAME chains resolving through trafficmanager.net and centralus.cloudapp.azure.com, ultimately resolving to 4.150.223.96.

Step 12 - Correlate DNS Resolution with TCP/TLS Traffic

Traffic involving 4.150.223.96 was located. A TCP connection from 192.168.1.7 (source port 50175) to 4.150.223.96 (destination port 443) was observed, carrying TLS 1.2-encrypted data - consistent with an HTTPS connection to a Microsoft cloud endpoint.

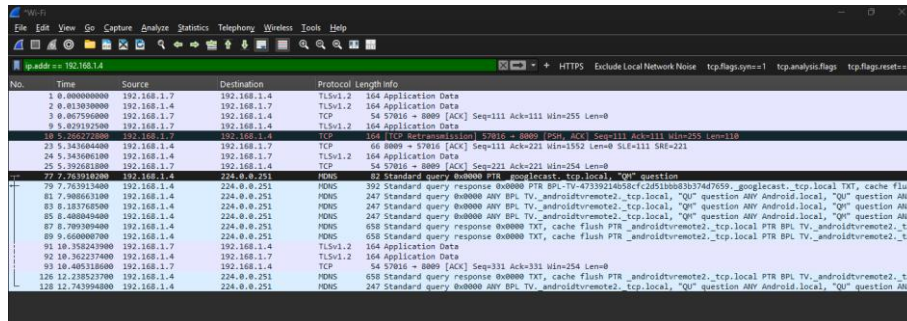
Step 13 - Examine an Additional TCP Connection

A separate TCP connection was observed: source 172.217.70.188 (port 5228) → destination 192.168.1.7 (port 60971). This was analyzed to demonstrate that an unfamiliar port number does not by itself indicate suspicious activity.

6. Detailed Traffic Investigation

This section presents a detailed, figure-by-figure walkthrough of every screenshot captured during the investigation. Each figure is interpreted technically and connected to the overall investigation narrative.

Figure 1 - Main Wireshark Capture Filtered by ip.addr == 192.168.1.4



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.1.7	192.168.1.4	TLSv1.2	164	Application Data
2	0.013030000	192.168.1.4	192.168.1.7	TLSv1.2	164	Application Data
3	0.067596000	192.168.1.7	192.168.1.4	TCP	54	57016 → 8009 [ACK] Seq=111 Ack=111 Win=255 Len=0
4	0.070192000	192.168.1.7	192.168.1.4	TLSv1.2	164	Application Data
10	5.260222000	192.168.1.7	192.168.1.4	TCP	164	TCP Retransmission 57016 → 8009 [PSH, ACK] Seq=111 Ack=111 Win=255 Len=160
23	5.343044000	192.168.1.4	192.168.1.7	TCP	60	8009 → 57016 [ACK] Seq=111 Ack=221 Win=1552 Len=0 SILENCE
24	5.345060000	192.168.1.4	192.168.1.7	TLSv1.2	164	Application Data
25	5.392681000	192.168.1.7	192.168.1.4	TCP	54	57016 → 8009 [ACK] Seq=221 Ack=221 Win=254 Len=0
77	7.763131000	192.168.1.4	224.0.0.251	MDNS	128	Standard query 0x0000 PTR BPL-TV-47339214058cf2d51b0d3b374d7659._googlecast._tcp.local TXT, cache flush
78	7.763131000	192.168.1.4	224.0.0.251	MDNS	128	Standard query response 0x0000 PTR BPL-TV-47339214058cf2d51b0d3b374d7659._googlecast._tcp.local TXT, cache flush
81	7.980661000	192.168.1.4	224.0.0.251	MDNS	247	Standard query 0x0000 ANY BPL-TV._androidtvremote2._tcp.local, "QR" question ANY Android.local, "QR" question ANY
83	8.183768000	192.168.1.4	224.0.0.251	MDNS	247	Standard query response 0x0000 ANY BPL-TV._androidtvremote2._tcp.local, "QR" question ANY Android.local, "QR" question ANY
85	8.480849000	192.168.1.4	224.0.0.251	MDNS	247	Standard query 0x0000 ANY BPL-TV._androidtvremote2._tcp.local, "QR" question ANY Android.local, "QR" question ANY
87	8.709049000	192.168.1.4	224.0.0.251	MDNS	658	Standard query response 0x0000 TXT, cache flush PTR _androidtvremote2._tcp.local PTR BPL-TV._androidtvremote2._tcp.local
89	9.600807000	192.168.1.4	224.0.0.251	MDNS	658	Standard query response 0x0000 TXT, cache flush PTR _androidtvremote2._tcp.local PTR BPL-TV._androidtvremote2._tcp.local
91	10.358243000	192.168.1.7	192.168.1.4	TLSv1.2	164	Application Data
92	10.362274000	192.168.1.4	192.168.1.7	TLSv1.2	164	Application Data
93	10.405316000	192.168.1.7	192.168.1.4	TCP	54	57016 → 8009 [ACK] Seq=331 Ack=331 Win=254 Len=0
126	12.238527000	192.168.1.4	224.0.0.251	MDNS	658	Standard query response 0x0000 TXT, cache flush PTR _androidtvremote2._tcp.local PTR BPL-TV._androidtvremote2._tcp.local
128	12.743994000	192.168.1.4	224.0.0.251	MDNS	247	Standard query 0x0000 ANY BPL-TV._androidtvremote2._tcp.local, "QR" question ANY Android.local, "QR" question ANY

Figure 1 - Wireshark main packet list filtered to show all traffic involving 192.168.1.4, with additional quick-filter buttons visible in the toolbar

What is shown:

This is the primary Wireshark capture window with the display filter `ip.addr == 192.168.1.4` applied (shown in the green filter bar at the top). The packet list shows traffic between 192.168.1.7, 192.168.1.4, and the mDNS multicast address 224.0.0.251. Protocols visible include TLSv1.2, TCP, and MDNS. The additional buttons in the toolbar ("HTTPS", "Exclude Local Network Noise", "tcp.flags.syn==1", "tcp.analysis.flags", "tcp.flags.reset==1") are pre-saved filter shortcuts.

Technical Interpretation:

The filter `ip.addr == 192.168.1.4` tells Wireshark to show only packets where either the source or destination IP address is 192.168.1.4. This immediately reveals two distinct types of traffic: TLS-encrypted application data being exchanged between 192.168.1.7 and 192.168.1.4, and MDNS (Multicast DNS) broadcast messages from 192.168.1.4 to 224.0.0.251.

Key observable details in the packet list:

- Packets 1 and 2: TLSv1.2 Application Data flowing in both directions between 192.168.1.7 and 192.168.1.4 - indicating an encrypted connection was already established.
- Packet 3: TCP acknowledgment on port 57016 → 8009, confirming the TCP session details.
- Packet 10: A TCP Retransmission on port 57016 → 8009 (highlighted in red), indicating a packet was resent - a normal TCP reliability mechanism.
- Packets 77 onward: MDNS queries from 192.168.1.4 for `_googlecast._tcp.local` - the first indication of what service this device is running.
- Packets 79, 89, 126: MDNS responses containing `"BPL TV._androidtvremote2._tcp.local"` - identifying the device as a BPL Android TV.

Why it matters:

This single screenshot establishes the investigation's core subject: 192.168.1.4 is an active device on the network that is both communicating with the local machine over TLS and advertising services using mDNS. These two observations drive the entire investigation forward.

Finding:

192.168.1.4 is an active network device involved in both encrypted TCP communication and local service discovery via mDNS. It advertises itself with names containing `"BPL TV"` and `"androidtvremote2"`, identifying it as a BPL Android TV device.

Figure 2 - mDNS Filtered Traffic and Packet Detail Pane

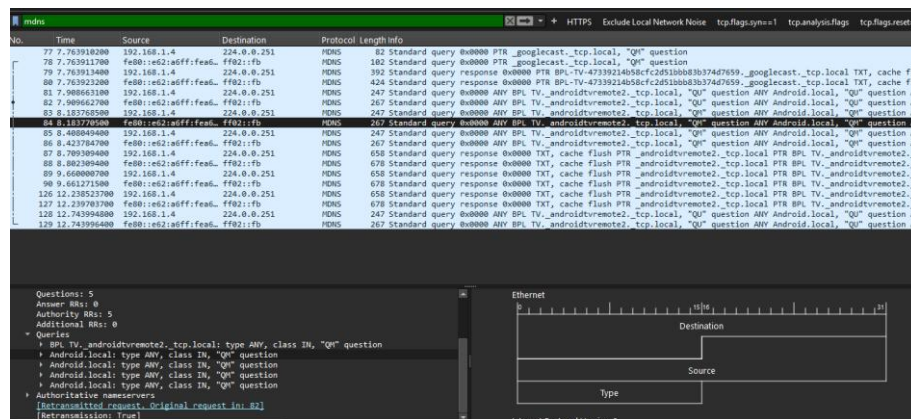


Figure 2 - Wireshark filtered to show only MDNS traffic (filter: mdns), with the packet detail pane expanded for a selected MDNS query packet

What is shown:

The display filter has been changed to mdns, showing only Multicast DNS packets. The packet list shows traffic from two sources: 192.168.1.4 (IPv4) and fe80::e62:a6ff:fea6... (an IPv6 link-local address), both sending to multicast addresses (224.0.0.251 for IPv4 mDNS, ff02::fb for IPv6 mDNS). The lower pane shows the decoded packet detail for the selected packet (Packet 84, highlighted in blue).

The detail pane shows:

- Questions: 5
- Answer RRs: 0
- Authority RRs: 5
- Queries including: BPL TV._androidtvremote2._tcp.local (type ANY, class IN, "QM" question) and Android.local (repeated queries)
- [Retransmission: True] - this packet was a retransmission of Packet 82

Technical Interpretation:

mDNS (Multicast DNS) is a protocol defined in RFC 6762 that allows devices on a local network to resolve hostnames and discover services without any central DNS server. When a device wants to find or announce a service, it sends a multicast packet to 224.0.0.251 (IPv4) or ff02::fb (IPv6), which all mDNS-capable devices on the subnet receive.

The query type "QM" stands for "Qu-Mode" (question with multicast), while "QU" in other packets stands for a unicast-preferred query. The query for "BPL TV._androidtvremote2._tcp.local" means a device is asking: "Is there anyone on this network advertising the Android TV Remote 2 service under the name BPL TV?" The repeated queries for "Android.local" are the device also trying to resolve its own hostname.

The IPv6 link-local address fe80::e62:a6ff:fea6... also sending mDNS queries confirms this is the same device using both IPv4 and IPv6 networking simultaneously - common on modern devices.

Why it matters:

This screenshot confirms that mDNS is the mechanism by which 192.168.1.4 is announcing itself to the local network. Without understanding mDNS, we would not be able to interpret why this device is broadcasting to 224.0.0.251. It also reveals the device's service names directly within the packet content.

Finding:

192.168.1.4 is actively using mDNS to advertise itself as "BPL TV" running the "_androidtvremote2._tcp" service. It is also querying for its own hostname "Android.local". Both IPv4 and IPv6 mDNS are in use.

Figure 3 - TCP Stream 0: Connection Between 192.168.1.7 and 192.168.1.4:8009

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.1.7	192.168.1.4	TLSv1.2	164	Application Data
2	0.013030000	192.168.1.4	192.168.1.7	TLSv1.2	164	Application Data
3	0.067596000	192.168.1.7	192.168.1.4	TCP	54	57016 → 8009 [ACK] Seq=111 Ack=111 Win=255 Len=0
4	0.079192000	192.168.1.7	192.168.1.4	TLSv1.2	164	Application Data
5	0.092922000	192.168.1.7	192.168.1.4	TCP	104	[TCP Retransmission] 57016 → 8009 (PSH, ACK) Seq=111 Ack=111 Win=255 Len=110
6	0.104922000	192.168.1.7	192.168.1.4	TCP	68	8009 → 57016 [ACK] Seq=111 Ack=221 Win=1552 Len=0 SLE=111 SRE=221
7	0.104922000	192.168.1.7	192.168.1.4	TLSv1.2	164	Application Data
8	0.104922000	192.168.1.7	192.168.1.4	TCP	54	57016 → 8009 [ACK] Seq=221 Ack=221 Win=254 Len=0
9	10.358243000	192.168.1.7	192.168.1.4	TLSv1.2	164	Application Data
10	10.362237000	192.168.1.7	192.168.1.4	TLSv1.2	164	Application Data
11	10.405310000	192.168.1.7	192.168.1.4	TCP	54	57016 → 8009 [ACK] Seq=331 Ack=331 Win=254 Len=0

Figure 3 - Wireshark filtered to tcp.stream eq 0, showing the complete TCP conversation between 192.168.1.7 (port 57016) and 192.168.1.4 (port 8009)

What is shown:

The display filter tcp.stream eq 0 isolates a single TCP conversation - Stream 0 - which contains all packets belonging to the connection between 192.168.1.7 (ephemeral source port 57016) and 192.168.1.4 (destination port 8009). The packet list shows TLSv1.2 Application Data packets interspersed with TCP control packets (ACKs, and one retransmission).

Technical Interpretation:

In Wireshark, each unique TCP connection is assigned a stream index starting from 0. The filter tcp.stream eq 0 shows all packets that belong to a specific TCP connection - regardless of direction. This is one of the most powerful ways to isolate and study a single conversation.

Port 57016 is an ephemeral port - a temporary, high-numbered port automatically chosen by the operating system when the client initiates a TCP connection. The destination port 8009 is the server-side port on 192.168.1.4. The presence of [ACK] packets confirms normal TCP operation (receiver acknowledging received data), while the [TCP Retransmission] at Packet 10 shows that one packet was resent because no acknowledgment was received within the expected timeout.

The TLSv1.2 Application Data packets confirm that once the TCP connection was established, all application-layer communication is protected by TLS encryption.

Why it matters:

This view isolates the specific connection under investigation. It confirms that 192.168.1.7 is the client connecting outward to 192.168.1.4 on port 8009, and that the session contains encrypted data. Knowing the stream number allows us to use the "Follow TCP Stream" feature in the next step.

Finding:

A TCP connection exists between 192.168.1.7 (source port 57016) and 192.168.1.4 (destination port 8009). The connection is active and carries TLS 1.2 encrypted application data. A TCP retransmission occurred, which is normal in real network captures.

Figure 4 - Follow TCP Stream: Encrypted Payload

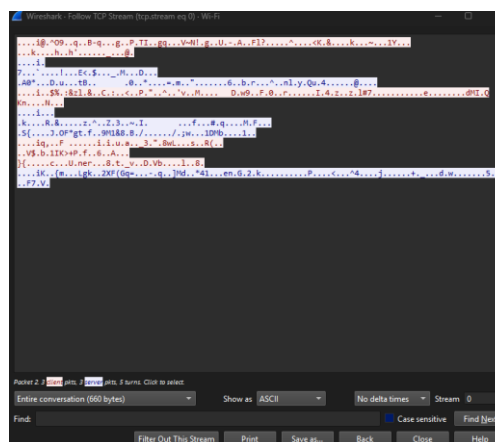


Figure 4 - Wireshark's "Follow TCP Stream" view of Stream 0, showing unreadable binary content due to TLS encryption (displayed in ASCII mode)

What is shown:

This is the output of Wireshark's "Follow TCP Stream" function applied to TCP Stream 0 (the 192.168.1.7 ↔ 192.168.1.4:8009 connection). The content is shown in ASCII mode. The display shows garbled, unreadable characters - a mixture of dots, symbols, and random-looking text. The status bar at the bottom reads: "Packet 2, 3 client pkts, 3 server pkts, 5 turns. Click to select." The total conversation size is 660 bytes. Stream 0 is selected.

Technical Interpretation:

The "Follow TCP Stream" feature reconstructs the full application-layer conversation from all packets in a TCP stream, presenting it as a readable stream of bytes. When an application uses unencrypted protocols like plain HTTP, this view would show clearly readable text - HTTP headers, HTML content, etc.

However, when TLS encryption is in use, the application data is encrypted before it is sent over the TCP connection. What Wireshark captures at the network level is already encrypted ciphertext - not the original plaintext. When the Follow TCP Stream view is shown in ASCII mode, it attempts to represent each byte as a printable character, which results in the meaningless garbled output visible here. The dots represent bytes whose values do not correspond to printable ASCII characters.

The fact that the stream shows 3 client packets and 3 server packets (5 turns) with 660 bytes total confirms that bidirectional data exchange occurred, but the actual content of that exchange is protected by TLS 1.2 and cannot be read without the private key.

Why it matters:

This screenshot visually demonstrates one of the most fundamental limitations in network traffic analysis: encryption prevents payload inspection. A network analyst can see that communication happened, how much data was exchanged, and in which direction - but cannot determine the meaning of the data. This is critically important for understanding both the power and the limitations of packet capture.

Finding:

The application-layer payload of the 192.168.1.7 ↔ 192.168.1.4:8009 TCP connection is encrypted by TLS 1.2 and therefore cannot be directly read or interpreted from the packet capture alone.

Figure 5 - TLS 1.2 Packet Detail: Application Data

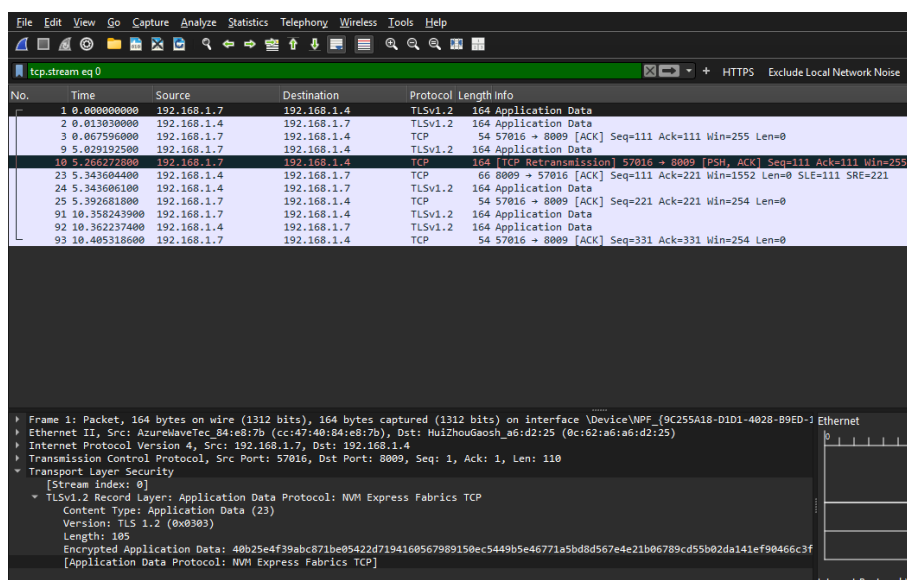


Figure 5 - Wireshark packet list filtered to tcp.stream eq 0, with the detail pane showing the full TLS 1.2 Application Data record for Packet 1

What is shown:

The main packet list continues to show TCP Stream 0. The packet detail pane (lower half) shows the full layer-by-layer breakdown of Packet 1. Key details visible include:

- Ethernet II: Source MAC - AzureWaveTec_84:e8:7b (cc:47:40:84:e8:7b) and Destination MAC - HuiZhouGaosh_a6:d2:25 (0c:62:a6:a6:d2:25)
- Internet Protocol Version 4: Src: 192.168.1.7, Dst: 192.168.1.4
- Transmission Control Protocol: Src Port: 57016, Dst Port: 8009, Seq: 1, Ack: 1, Len: 110
- Transport Layer Security: [Stream index: 0], TLSv1.2 Record Layer: Application Data Protocol: NVM Express Fabrics TCP, Content Type: Application Data (23), Version: TLS 1.2 (0x0303), Length: 105
- Encrypted Application Data:
40b25e4f39abc871be05422d7194160567989150ec5449b5e46771a5bd8d567e4e21b06789cd55b02da141ef90466c3f...

Technical Interpretation:

This screenshot beautifully illustrates the layered structure of the TCP/IP stack as it appears in a real packet. Reading from bottom to top of the protocol stack:

- At the Ethernet layer, the MAC addresses identify the hardware interfaces. "AzureWaveTec" in the MAC vendor name suggests the Wi-Fi adapter is made by AzureWave - a common OEM chipset manufacturer.
- At the IP layer, the source and destination IP addresses are confirmed: 192.168.1.7 and 192.168.1.4.
- At the TCP layer, the source port 57016 and destination port 8009 are confirmed, along with sequence numbers used for reliable delivery.
- At the TLS layer, the version is confirmed as TLS 1.2 (0x0303 is the TLS 1.2 version code). The Content Type 23 means "Application Data" in TLS terminology. The encrypted payload itself is shown as a hex string - this is the actual ciphertext.

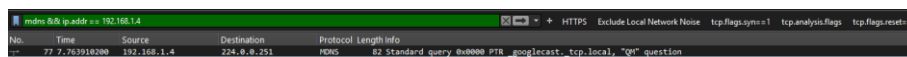
Why it matters:

This is the most technically detailed screenshot in the investigation. It confirms every element of the network stack involved in this communication. It also visually demonstrates how TLS sits inside TCP - the encrypted data is the TCP payload, but it cannot be decrypted without the server's private key.

Finding:

Packet 1 of TCP Stream 0 carries 105 bytes of TLS 1.2-encrypted Application Data from 192.168.1.7 (port 57016) to 192.168.1.4 (port 8009). The encryption is confirmed at version TLS 1.2 (0x0303), and the ciphertext is visible but uninterpretable without decryption keys.

Figure 6 - mDNS and IP Filter: Google Cast Query Visible



No.	Time	Source	Destination	Protocol	Length	Info
77	7.763918280	192.168.1.4	224.0.0.251	MDNS	82	Standard query 0x0000 PTR _googlecast._tcp.local, "QM" question

Figure 6 - Wireshark filtered to show mDNS traffic from 192.168.1.4 (filter: `mdns && ip.addr == 192.168.1.4`), showing the `_googlecast._tcp.local` query

What is shown:

This screenshot shows a narrow strip of the Wireshark packet list with the combined filter `mdns && ip.addr == 192.168.1.4` applied. The visible packet is Packet 77, showing 192.168.1.4 sending an MDNS Standard query to 224.0.0.251 - specifically a PTR (pointer) query for `_googlecast._tcp.local` with QM (multicast) mode.

Technical Interpretation:

The filter `mdns && ip.addr == 192.168.1.4` restricts the view to only mDNS packets that involve 192.168.1.4, removing all other traffic. A PTR record query in mDNS is a "service discovery" query - the device is asking the network: "Who on this network provides the `_googlecast._tcp` service?"

The `_googlecast._tcp.local` service name is Google's proprietary service name for the Google Cast protocol - the technology used by Chromecast devices and any device (such as a smart TV with Chromecast built in)

that allows other devices to "cast" content to it. The .local suffix indicates this is a local-network mDNS name, not a public DNS name.

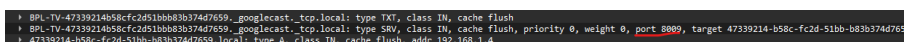
Why it matters:

This packet is pivotal: it is the first direct evidence that the device at 192.168.1.4 is involved with the Google Cast service. This is what connects the "BPL TV" Android device identity (from Figure 2) with the specific service it is advertising (Google Cast). The next step is to find the SRV record response that tells us which TCP port this service is running on.

Finding:

192.168.1.4 is querying for `_googlecast._tcp.local` via mDNS, confirming it is a Google Cast-capable device. This directly leads to the port-8009 relationship established in Figure 7.

Figure 7 - mDNS SRV Record: Google Cast Bound to Port 8009



A screenshot of a Wireshark packet detail pane showing three DNS resource records. The first record is a TXT record for `_googlecast._tcp.local` with type IN and a cache flush. The second record is an SRV record for `_googlecast._tcp.local` with type IN, class IN, priority 0, weight 0, port 8009, and target `47339214-b58c-fc2d-51bb-b83b374d7659.local`. The third record is an A record for `47339214-b58c-fc2d-51bb-b83b374d7659.local` with type IN, class IN, and address 192.168.1.4. The port number 8009 is highlighted in red in the original image.

Figure 7 - Wireshark packet detail showing the mDNS response containing the Google Cast SRV record with port 8009 and A record resolving to 192.168.1.4

What is shown:

This screenshot shows three DNS resource records from the packet detail pane of an mDNS response:

- BPL-TV-47339214b58cfc2d51bbb83b374d7659._googlecast._tcp.local: type TXT, class IN, cache flush (a TXT record with service metadata)
- BPL-TV-47339214b58cfc2d51bbb83b374d7659._googlecast._tcp.local: type SRV, class IN, cache flush, priority 0, weight 0, port 8009, target 47339214-b58c-fc2d-51bb-b83b374d7659.local (the SRV record identifying port 8009)
- 47339214-b58c-fc2d-51bb-b83b374d7659.local: type A, class IN, cache flush, addr 192.168.1.4 (an A record resolving the device hostname to its IP address)

The port number 8009 is underlined/highlighted in the screenshot.

Technical Interpretation:

This screenshot contains the most critical evidence in the entire investigation. Let us break down what each record type means:

- TXT record: Carries arbitrary text metadata about the service (often contains version information, device capabilities, etc.).
- SRV record (Service Record): This is the key record. An SRV record tells a client not just that a service exists, but exactly where to connect to it: the hostname (target) and the TCP/UDP port number. Here, it states: "The Google Cast service runs at port 8009 on the host 47339214-b58c-fc2d-51bb-b83b374d7659.local."
- A record: Resolves the hostname given in the SRV record ("47339214-b58c-fc2d-51bb-b83b374d7659.local") to the IP address 192.168.1.4.

Combining the SRV and A records: Google Cast runs at port 8009 on 192.168.1.4. This is how the port-service relationship is established from captured evidence, not from memorizing a port number table.

Why it matters:

This is the evidentiary link between the service name (`_googlecast._tcp`) and the TCP port (8009) observed in Figures 3–5. Without this SRV record, we could only observe that port 8009 was in use and speculate about what service it carried. With this record, we know from the captured traffic itself that port 8009 is where the Google Cast service lives on 192.168.1.4.

Finding:

The mDNS SRV record captured in this packet directly and definitively establishes that the Google Cast service on 192.168.1.4 operates on TCP port 8009. The A record confirms that the SRV target hostname resolves to 192.168.1.4. This is evidence-based port identification, not assumption.

Figure 8 - TLS Handshake Filter Returns No Results

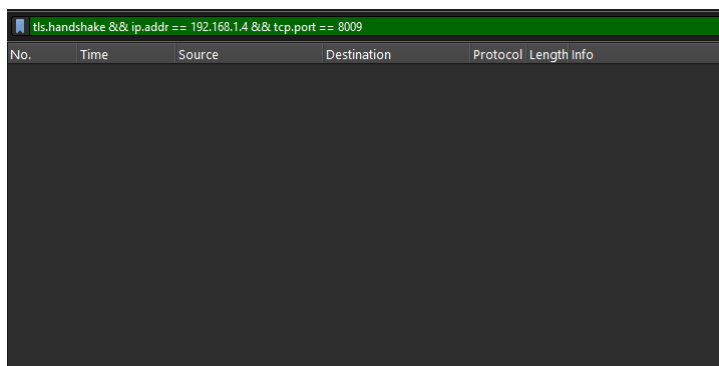


Figure 8 - Wireshark filter "tls.handshake && ip.addr == 192.168.1.4 && tcp.port == 8009" returning an empty packet list

What is shown:

The Wireshark filter bar shows the filter: `tls.handshake && ip.addr == 192.168.1.4 && tcp.port == 8009`. The packet list area is completely empty - no packets are shown. The filter bar is highlighted in green (valid syntax) but no matching packets exist in the capture.

Technical Interpretation:

A TLS handshake is the initial negotiation phase of a TLS connection, during which the client and server exchange messages to agree on encryption parameters, authenticate identities (via certificates), and establish shared encryption keys. The TLS handshake produces specific packet types: ClientHello, ServerHello, Certificate, Server Hello Done, Client Key Exchange, Change Cipher Spec, and Finished messages.

The filter `tls.handshake` searches for these specific handshake message types. When combined with `ip.addr == 192.168.1.4` and `tcp.port == 8009`, it searches specifically for TLS handshake packets involving the Google Cast port on 192.168.1.4.

The empty result means: no TLS handshake packets for this specific connection were present in the captured traffic window. This is a very important finding - but it must be interpreted carefully:

- The TLS connection between 192.168.1.7 and 192.168.1.4:8009 is clearly established (we see TLS Application Data in Figures 3–5).
- The handshake must have occurred before the capture started - the connection was already in progress when Wireshark began recording.
- This does NOT mean no handshake occurred. It means the handshake was not captured.

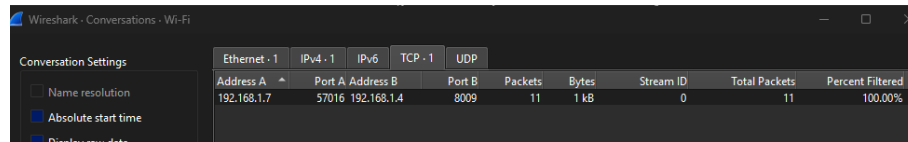
Why it matters:

This screenshot teaches one of the most important lessons in packet capture analysis: the absence of a packet in a capture does not mean that event never happened. Network captures are snapshots of time. Events that occurred before or after the capture window - or on other interfaces - will not appear.

Finding:

The TLS handshake for the 192.168.1.7 ↔ 192.168.1.4:8009 connection was not captured in this traffic file. The connection was already established when the capture began. This is a limitation of the capture, not evidence that no handshake occurred.

Figure 9 - TCP Conversation Statistics



The image shows the Wireshark 'Conversations' window with the 'TCP' tab selected. It displays a single conversation between 192.168.1.7 (Port A: 57016) and 192.168.1.4 (Port B: 8009). The statistics show 11 packets, 1 KB of data, and 0 stream ID. The 'Percent Filtered' is 100.00%.

Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Total Packets	Percent Filtered
192.168.1.7	57016	192.168.1.4	8009	11	1 KB	0	11	100.00%

Figure 9 - Wireshark Conversations window showing the TCP tab with one conversation: 192.168.1.7:57016 ↔ 192.168.1.4:8009, 11 packets, 1 KB

What is shown:

This is the Wireshark Statistics → Conversations panel, with the TCP tab selected (shown as "TCP · 1", meaning one TCP conversation). The table shows one row with the following details:

- Address A: 192.168.1.7, Port A: 57016
- Address B: 192.168.1.4, Port B: 8009
- Packets: 11
- Bytes: 1 KB
- Stream ID: 0
- Total Packets: 11
- Percent Filtered: 100.00%

Technical Interpretation:

The Conversations panel aggregates all packets belonging to each unique network connection and summarizes them. For TCP, each conversation is identified by the four-tuple: (Source IP, Source Port, Destination IP, Destination Port). The "Percent Filtered: 100.00%" means that, with the current display filter active (tcp.stream eq 0), all 11 packets of this conversation are visible.

11 packets for 1 KB of total data is consistent with what was observed in the packet list: several TLS Application Data frames (each carrying 105 bytes of payload), TCP ACK frames (which carry no payload and are 54 bytes each), and one retransmission. The conversation is bidirectional - both sides have sent data.

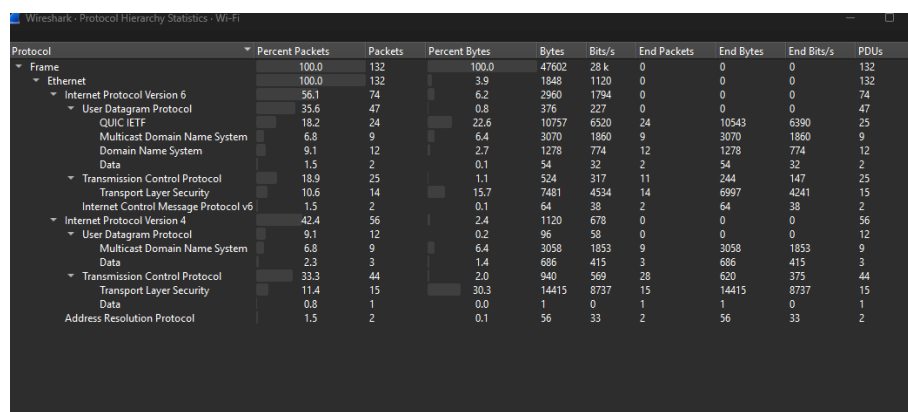
Why it matters:

The Conversations panel provides a quick summary that is useful for situational awareness. Rather than reading through dozens of individual packets, this view immediately shows how many unique TCP conversations exist, how much data each one carried, and on which ports. In a larger capture with many connections, this view would be essential for prioritizing which conversations to investigate further.

Finding:

The 192.168.1.7 ↔ 192.168.1.4:8009 TCP conversation comprised 11 total packets and approximately 1 KB of data. This is the only TCP conversation in the filtered view, confirming it is the primary connection under investigation.

Figure 10 - Protocol Hierarchy Statistics



The image shows the Wireshark 'Protocol Hierarchy Statistics' window. It provides a detailed breakdown of network protocols and their statistics. The table includes columns for Protocol, Percent Packets, Packets, Percent Bytes, Bytes, Bits/s, End Packets, End Bytes, End Bits/s, and PDUs.

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	132	100.0	47602	28 k	0	0	0	132
Ethernet	100.0	132	3.9	1848	1120	0	0	0	132
Internet Protocol Version 6	56.1	74	6.2	2960	1794	0	0	0	74
User Datagram Protocol	35.6	47	0.8	376	227	0	0	0	47
QUIC IETF	18.2	24	22.6	10757	6520	24	10543	6390	25
Multicast Domain Name System	6.8	9	6.4	3070	1860	9	3070	1860	9
Domain Name System	9.1	12	2.7	1278	774	12	1278	774	12
Data	1.5	2	0.1	54	32	2	54	32	2
Transmission Control Protocol	18.9	25	1.1	524	317	11	244	147	25
Transport Layer Security	10.6	14	15.7	7481	4534	14	6997	4241	15
Internet Control Message Protocol v6	1.5	2	0.1	64	38	2	64	38	2
Internet Protocol Version 4	42.4	56	2.4	1120	678	0	0	0	56
User Datagram Protocol	9.1	12	0.2	96	58	0	0	0	12
Multicast Domain Name System	6.8	9	6.4	3058	1853	9	3058	1853	9
Data	2.3	3	1.4	686	415	3	686	415	3
Transmission Control Protocol	33.3	44	2.0	940	569	28	620	375	44
Transport Layer Security	11.4	15	30.3	14415	8737	15	14415	8737	15
Data	0.8	1	0.0	1	0	1	1	0	1
Address Resolution Protocol	1.5	2	0.1	56	33	2	56	33	2

Figure 10 - Wireshark Protocol Hierarchy Statistics window showing the full breakdown of all 132 captured frames by protocol

What is shown:

The Protocol Hierarchy Statistics window shows all protocols observed in the entire capture (132 total frames, 47,602 bytes). The hierarchy is organized as a tree, showing how protocols nest within each other. Key rows include:

- Frame: 100% (132 packets, 47,602 bytes)
- Ethernet: 100% (132 packets)
- Internet Protocol Version 6 (IPv6): 56.1% (74 packets, 2,960 bytes)
- User Datagram Protocol (UDP) under IPv6: 35.6% (47 packets)
- QUIC IETF: 18.2% (24 packets, 10,757 bytes) - the largest single protocol by bytes
- Multicast Domain Name System (mDNS): 6.8% (9 packets)
- Domain Name System (DNS): 9.1% (12 packets)
- Transmission Control Protocol (TCP) under IPv6: 18.9% (25 packets)
- Transport Layer Security (TLS) under IPv6/TCP: 10.6% (14 packets, 7,481 bytes)
- ICMPv6: 1.5% (2 packets)
- Internet Protocol Version 4 (IPv4): 42.4% (56 packets, 1,120 bytes)
- UDP under IPv4: 9.1% (12 packets)
- mDNS under IPv4: 6.8% (9 packets)
- TCP under IPv4: 33.3% (44 packets)
- TLS under IPv4/TCP: 11.4% (15 packets, 14,415 bytes)
- Address Resolution Protocol (ARP): 1.5% (2 packets)

Technical Interpretation:

The Protocol Hierarchy view is one of the most informative initial analysis steps for any packet capture. Rather than diving into individual packets, it gives an instant high-level picture of "what kind of traffic is this capture mostly made of?"

Key observations from this capture:

- More than half the traffic (56.1%) is IPv6, which is common on modern Wi-Fi networks that support both IPv4 and IPv6 simultaneously.
- QUIC (18.2% of packets but 22.6% of bytes - 10,757 bytes) is the largest single protocol by data volume. QUIC is used by Google services (including YouTube, Google Search, and Gmail) and represents background internet traffic. It is encrypted by default.
- TLS traffic appears under both IPv4 (15 packets, 14,415 bytes) and IPv6 (14 packets, 7,481 bytes), confirming widespread encryption in this capture.
- mDNS appears under both IPv4 (9 packets) and IPv6 (9 packets), consistent with what was observed during the device identification phase.
- ARP (2 packets) represents local address resolution traffic - devices mapping IP addresses to MAC addresses.

Why it matters:

The Protocol Hierarchy establishes the context for the entire investigation. Knowing that the capture contains QUIC, TLS, mDNS, and DNS helps explain why some traffic is encrypted (QUIC and TLS), why some is local (mDNS), and why some involves name resolution (DNS). Without this overview, the investigation would lack context.

Finding:

The 132-packet capture contains a rich mix of protocols typical of a modern home or office Wi-Fi network: heavily encrypted traffic (QUIC and TLS), local service discovery (mDNS), name resolution (DNS), and standard TCP connections. No unusual or clearly malicious protocol patterns are visible.

Figure 11 - DNS Response for mobile.events.data.microsoft.com

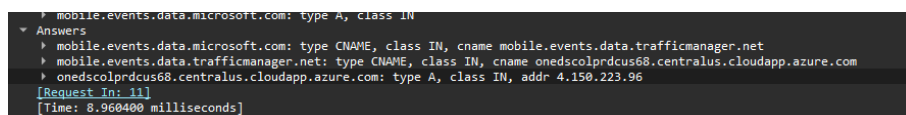


Figure 11 - Wireshark packet detail showing the DNS response for mobile.events.data.microsoft.com, resolving through CNAME chain to 4.150.223.96

What is shown:

This screenshot shows the "Answers" section of a DNS response packet. The following DNS resource records are visible in the answer section:

- mobile.events.data.microsoft.com: type CNAME, class IN, cname mobile.events.data.trafficmanager.net
- mobile.events.data.trafficmanager.net: type CNAME, class IN, cname onedscolprdcus68.centralus.cloudapp.azure.com
- onedscolprdcus68.centralus.cloudapp.azure.com: type A, class IN, addr 4.150.223.96

At the bottom: [Request In: 11] and [Time: 8.960400 milliseconds]

Technical Interpretation:

This DNS response illustrates a CNAME chain - a common DNS design pattern used by large organizations to manage their services flexibly. Let's trace the resolution step by step:

- Step 1: The client asked for mobile.events.data.microsoft.com. The DNS response says: "That hostname is an alias (CNAME) for mobile.events.data.trafficmanager.net."
- Step 2: The response also resolves mobile.events.data.trafficmanager.net: "That is an alias (CNAME) for onedscolprdcus68.centralus.cloudapp.azure.com."
- Step 3: The final A record resolves the Azure hostname to an actual IP address: 4.150.223.96.

A CNAME (Canonical Name) record is a DNS record that creates an alias from one hostname to another. A records (Address records) map a hostname to an IPv4 address. The response time of 8.96 milliseconds suggests this was an external DNS query (not served from local cache).

The domain name mobile.events.data.microsoft.com is a Microsoft telemetry / diagnostic endpoint - a service that Microsoft applications use to send usage and diagnostic data to Microsoft's cloud infrastructure. The trafficmanager.net intermediate name is Microsoft Azure Traffic Manager, a load-balancing service. The centralus.cloudapp.azure.com name confirms the endpoint is hosted in Microsoft Azure's Central US region.

Why it matters:

This DNS response reveals that a device on the network (most likely 192.168.1.7) was communicating with Microsoft telemetry infrastructure. Importantly, a DNS response alone does not prove that an actual TCP connection was established - it only proves the IP address was resolved. The next figure will show whether a corresponding connection was actually observed.

Finding:

A DNS query for mobile.events.data.microsoft.com was captured and resolved to 4.150.223.96 via a CNAME chain through Microsoft Azure infrastructure. This IP address will be correlated with observed TCP/TLS traffic in Figure 12.

Figure 12 - TCP Connection to 4.150.223.96 on Port 443

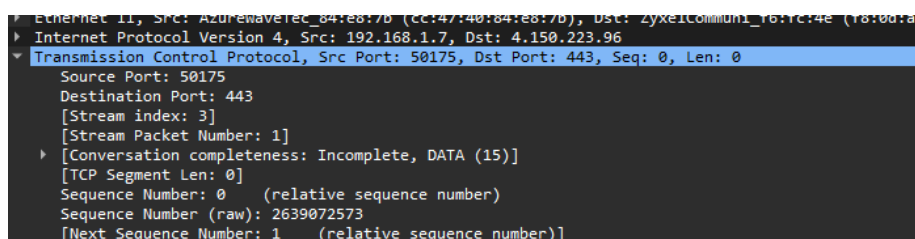


Figure 12 - Wireshark TCP packet detail showing a connection from 192.168.1.7 (port 50175) to 4.150.223.96 (port 443), with TLS 1.2 confirmed

What is shown:

This screenshot shows the packet detail pane for a TCP packet. The relevant fields are:

- Ethernet II: Source - AzureWaveTec_84:e8:7b, Destination - ZyXelCommuni_f6:fc:4e
- Internet Protocol Version 4: Src: 192.168.1.7, Dst: 4.150.223.96
- Transmission Control Protocol:
 - Source Port: 50175
 - Destination Port: 443
 - [Stream index: 3]
 - [Stream Packet Number: 1]
 - [Conversation completeness: Incomplete, DATA (15)]
 - TCP Segment Len: 0
 - Sequence Number: 0 (relative sequence number)

Technical Interpretation:

This packet shows a TCP connection from 192.168.1.7 (the local machine, using ephemeral source port 50175) to 4.150.223.96 (destination port 443). Port 443 is the standard HTTPS/TLS port. The Destination MAC address "ZyXelCommuni_f6:fc:4e" is the local router/gateway - all outgoing internet traffic would be addressed to the router MAC at Layer 2, while the IP destination remains the internet address.

The note "[Conversation completeness: Incomplete, DATA (15)]" means Wireshark observed data packets in this conversation but did not capture the complete session (the SYN, FIN, and likely some packets are missing). Stream index 3 means this is the fourth unique TCP connection observed in the capture (streams are indexed from 0).

The presence of this TCP connection to 4.150.223.96:443 directly following the DNS resolution in Figure 11 demonstrates the complete DNS-to-connection workflow: the device resolved the IP address via DNS, then initiated a TCP connection to that address on port 443 (HTTPS), over which TLS-encrypted data was exchanged.

Why it matters:

This screenshot closes the investigative loop opened in Figure 11. The DNS response (Figure 11) proved that 4.150.223.96 was resolved. This screenshot proves that an actual TCP connection to that IP address on port 443 was observed. Together, they demonstrate a complete communication event: name resolution → address obtained → connection established → encrypted communication.

Finding:

A TCP connection from 192.168.1.7 (source port 50175) to 4.150.223.96 (destination port 443) was captured, corresponding to the DNS resolution of mobile.events.data.microsoft.com observed in Figure 11. This is consistent with a Microsoft application on the local machine sending diagnostic data to Azure infrastructure over HTTPS/TLS.

Figure 13 - Additional TCP Connection: 172.217.70.188 → 192.168.1.7:60971

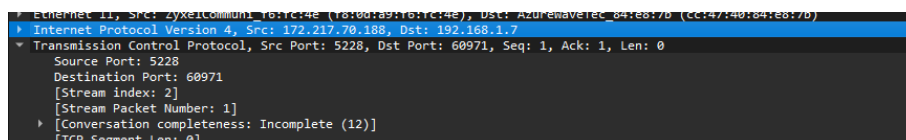


Figure 13 - Wireshark TCP packet detail showing a connection from 172.217.70.188 (port 5228) to 192.168.1.7 (port 60971)

What is shown:

This screenshot shows the packet detail pane for another TCP packet. The relevant fields are:

- Ethernet II: Source - ZyXelCommuni_f6:fc:4e (router), Destination - AzureWaveTec_84:e8:7b (local machine)
- Internet Protocol Version 4: Src: 172.217.70.188, Dst: 192.168.1.7
- Transmission Control Protocol:
 - Source Port: 5228
 - Destination Port: 60971
 - [Stream index: 2]
 - [Stream Packet Number: 1]
 - [Conversation completeness: Incomplete (12)]
 - TCP Segment Len: 0

Technical Interpretation:

The source IP address 172.217.70.188 is a Google IP address (part of Google's public IP range). Port 5228 is a port used by Google's FCM (Firebase Cloud Messaging) service, which is Google's push notification infrastructure used by Android and many applications to deliver notifications and messages. The destination port 60971 is an ephemeral port on 192.168.1.7.

The direction of this packet is incoming - the source is the internet (172.217.70.188) and the destination is the local device (192.168.1.7). This is a normal pattern for persistent TCP connections to notification services: the local device establishes a long-lived connection to the Google service, which then sends data (push notifications) to the local device.

Port 5228 is not a widely memorized "well-known" port like 443 (HTTPS) or 80 (HTTP). However, its unusual number alone should not be treated as suspicious. Context matters: the source IP belongs to Google infrastructure, the destination is a local device, and the connection pattern is consistent with legitimate push notification services used by apps.

Why it matters:

This screenshot serves as an important teaching example. It demonstrates that an unfamiliar port number is not by itself evidence of malicious activity. Proper investigation requires considering the source IP address, the direction of traffic, the protocol context, and correlation with other evidence before drawing conclusions. Jumping to the conclusion that "port 5228 is suspicious" without this context would be poor analysis.

Finding:

An incoming TCP connection from 172.217.70.188 (a Google IP) on source port 5228 to 192.168.1.7 on ephemeral port 60971 was observed. This is consistent with Google push notification (FCM) infrastructure communicating with an application on the local machine. The unfamiliar port number 5228 is not evidence of malicious activity - context identifies it as a legitimate Google service port.

7. Specific Investigation Findings

7.1 Local Device Identification

The investigation began by applying the Wireshark filter `ip.addr == 192.168.1.4`, which isolated all traffic involving a second device on the local network. Two categories of traffic were immediately apparent: TLS-encrypted TCP communication between 192.168.1.7 and 192.168.1.4, and MDNS broadcast traffic from 192.168.1.4 to the multicast address 224.0.0.251.

Within the mDNS packets, the Info column revealed service names containing "BPL TV._androidtvremote2._tcp.local". The mDNS protocol carries these service names as plaintext, which is what makes it useful for service discovery - and also what made identification possible from the packet capture. BPL is an Indian electronics manufacturer, and the "_androidtvremote2._tcp" service type is the Android TV remote control service. This combination positively identifies 192.168.1.4 as a BPL-brand Android smart television.

7.2 mDNS Investigation

Multicast DNS (mDNS) is a protocol that enables zero-configuration networking on local networks. When a device wants to advertise a service or discover services from others, it sends DNS-format packets to the multicast address 224.0.0.251 (IPv4) or `ff02::fb` (IPv6). Every device on the local network receives these packets.

In this capture, 192.168.1.4 was the primary mDNS participant from among the local devices. It sent both queries (asking about services) and responses (announcing its own services). The mDNS traffic revealed several service names associated with 192.168.1.4:

- `_androidtvremote2._tcp.local` - Android TV Remote 2 service
- `_googlecast._tcp.local` - Google Cast service
- `Android.local` - the device's own hostname query

Both IPv4 (from 192.168.1.4 to 224.0.0.251) and IPv6 (from `fe80::e62:a6ff:fea6...` to `ff02::fb`) mDNS traffic was observed, which is normal for dual-stack devices.

7.3 Google Cast Service Investigation

Within the mDNS traffic, a specific PTR record query for `_googlecast._tcp.local` was observed (Figure 6). This is the standardized DNS service type for Google Cast. The corresponding mDNS response (Figure 7) contained three record types that together fully describe the service:

- A TXT record containing service metadata for BPL-TV-47339214b58cfc2d51bbb83b374d7659._googlecast._tcp.local
- An SRV record stating: priority 0, weight 0, port 8009, target 47339214-b58c-fc2d-51bb-b83b374d7659.local
- An A record resolving 47339214-b58c-fc2d-51bb-b83b374d7659.local to 192.168.1.4

This three-record combination is the standard DNS-SD (DNS-based Service Discovery) mechanism. The SRV record is what provides the TCP port. The A record is what provides the IP address. Together, they give the complete connection target: 192.168.1.4:8009. This is how the port-service relationship was established from evidence in the capture, not from consulting a port number database.

7.4 TCP Port 8009 Investigation

With the Google Cast service established as operating on TCP port 8009 at 192.168.1.4 (from the SRV record), the TCP conversation was examined. The filter `tcp.stream eq 0` isolated the specific connection, revealing:

- Client (initiator): 192.168.1.7, ephemeral source port 57016
- Server (listener): 192.168.1.4, destination port 8009
- Total packets: 11

- Total bytes: approximately 1 KB
- Stream ID: 0 (the first TCP conversation in the capture)

The connection was bidirectional: both client-to-server (TLS Application Data going to port 8009) and server-to-client (TLS Application Data returning from port 8009) packets were observed. TCP ACK packets from both sides confirmed normal TCP operation. One TCP Retransmission (Packet 10) was observed, which is a normal TCP reliability mechanism.

7.5 TLS Investigation

The TLS investigation involved two key steps: using the Follow TCP Stream feature and attempting to filter for the TLS handshake.

Follow TCP Stream (Figure 4):

When the "Follow TCP Stream" function was applied to TCP Stream 0 (the 192.168.1.7 ↔ 192.168.1.4:8009 connection), the output was completely unreadable - a stream of garbled ASCII characters representing encrypted binary data. This is the expected behavior when TLS is in use. The stream showed 3 client packets and 3 server packets across 5 turns of data exchange, totaling 660 bytes, all encrypted.

TLS Handshake Filter (Figure 8):

The filter `tls.handshake && ip.addr == 192.168.1.4 && tcp.port == 8009` was applied to search for the TLS handshake of this connection. The result was an empty packet list - no handshake packets were found. This means the TLS handshake occurred before the Wireshark capture began. The connection was already established by the time recording started. This does not mean no handshake happened - it means this particular capture did not include it.

TLS Version (Figure 5):

From the packet detail pane of Packet 1, the TLS version was confirmed as TLS 1.2 (version code 0x0303). The Content Type 23 (Application Data) was observed. The encrypted payload was visible as a hex string but could not be decrypted without the server's private key.

7.6 TCP Conversation Statistics

The Wireshark Statistics → Conversations → TCP tab (Figure 9) confirmed the following for the active TCP conversation:

- Address A: 192.168.1.7, Port A: 57016
- Address B: 192.168.1.4, Port B: 8009
- Total Packets: 11
- Total Bytes: ~1 KB
- Stream ID: 0

This was the only TCP conversation visible with the current filter active, confirming that the Google Cast connection was the primary TCP session under investigation in the filtered view.

7.7 Protocol Hierarchy

The Protocol Hierarchy Statistics (Figure 10) provided an overview of the entire 132-packet capture. Key observations:

- IPv6 traffic (56.1%) outnumbered IPv4 traffic (42.4%) - reflecting modern dual-stack networking.
- QUIC (18.2% of packets, 22.6% of bytes) was the dominant protocol by data volume - consistent with Google service traffic running in the background.
- TLS was present under both IPv4 and IPv6, confirming widespread encryption throughout the capture.
- mDNS appeared under both IPv4 (9 packets) and IPv6 (9 packets), matching the observed device discovery traffic.
- DNS (12 packets) included the resolution queries observed in Figure 11.
- ARP (2 packets) represented local address resolution - normal background traffic.

Protocol hierarchy analysis is valuable because it immediately tells an analyst what kind of traffic dominates a capture, where to focus investigation effort, and whether any unexpected protocols are present.

7.8 DNS Investigation

A DNS query for `mobile.events.data.microsoft.com` was located in the capture. The DNS response (Figure 11) returned a CNAME chain:

- `mobile.events.data.microsoft.com` → CNAME → `mobile.events.data.trafficmanager.net`
- `mobile.events.data.trafficmanager.net` → CNAME → `onedscolprdcus68.centralus.cloudapp.azure.com`
- `onedscolprdcus68.centralus.cloudapp.azure.com` → A record → `4.150.223.96`

An A record (Address record) is the fundamental DNS record type that maps a hostname to an IPv4 address. The CNAME chain here reflects Microsoft's use of Azure Traffic Manager (`trafficmanager.net`) for load balancing and geographic routing, with the actual endpoint hosted on Azure infrastructure (`centralus.cloudapp.azure.com`).

Important limitation: the DNS response proves that the IP address `4.150.223.96` was resolved. It does not, by itself, prove that an application actually connected to that address. Correlation with TCP traffic (Section 7.9) is needed to establish that.

7.9 DNS → TCP/TLS Correlation

After identifying the resolved IP address `4.150.223.96` from the DNS response, TCP traffic involving that IP was located in the capture. Figure 12 shows a TCP packet with:

- Source: `192.168.1.7`, Source Port: `50175`
- Destination: `4.150.223.96`, Destination Port: `443`
- Stream index: `3`

Destination port `443` is the standard port for HTTPS - HTTP over TLS. This TCP connection, following the DNS resolution, demonstrates the complete communication workflow: a Microsoft application on `192.168.1.7` resolved the domain name via DNS (Figure 11), obtained the IP address `4.150.223.96`, and then established an HTTPS/TLS connection to that address on port `443` (Figure 12).

This correlation between a DNS resolution and a subsequent TCP connection is one of the most important techniques in traffic analysis. A DNS query shows intent; a TCP connection shows action.

7.10 Additional TCP Connection - Port 5228

Figure 13 documents a separate TCP connection: source `172.217.70.188` (port `5228`) incoming to `192.168.1.7` (destination port `60971`). The source IP `172.217.70.188` belongs to Google's infrastructure. Port `5228` is associated with Google's Firebase Cloud Messaging (FCM) push notification service.

This connection is incoming to the local machine - meaning a Google server is sending data to the local device over an established connection. This is consistent with how push notification systems work: the application on the local device maintains a persistent connection to Google's notification service, which then delivers notifications as needed.

The key lesson from this finding is that an unfamiliar port number (`5228`) should not be treated as evidence of suspicious activity without context. Proper analysis requires:

- Identifying the source IP and its ownership (Google infrastructure in this case)
- Considering the direction of the traffic (incoming from a legitimate server)
- Examining the context of other traffic from the same IP
- Correlating with DNS queries to see if the IP was resolved

Jumping to a conclusion of "this port is suspicious" based solely on the port number would be incorrect analysis.

8. Protocol Explanation

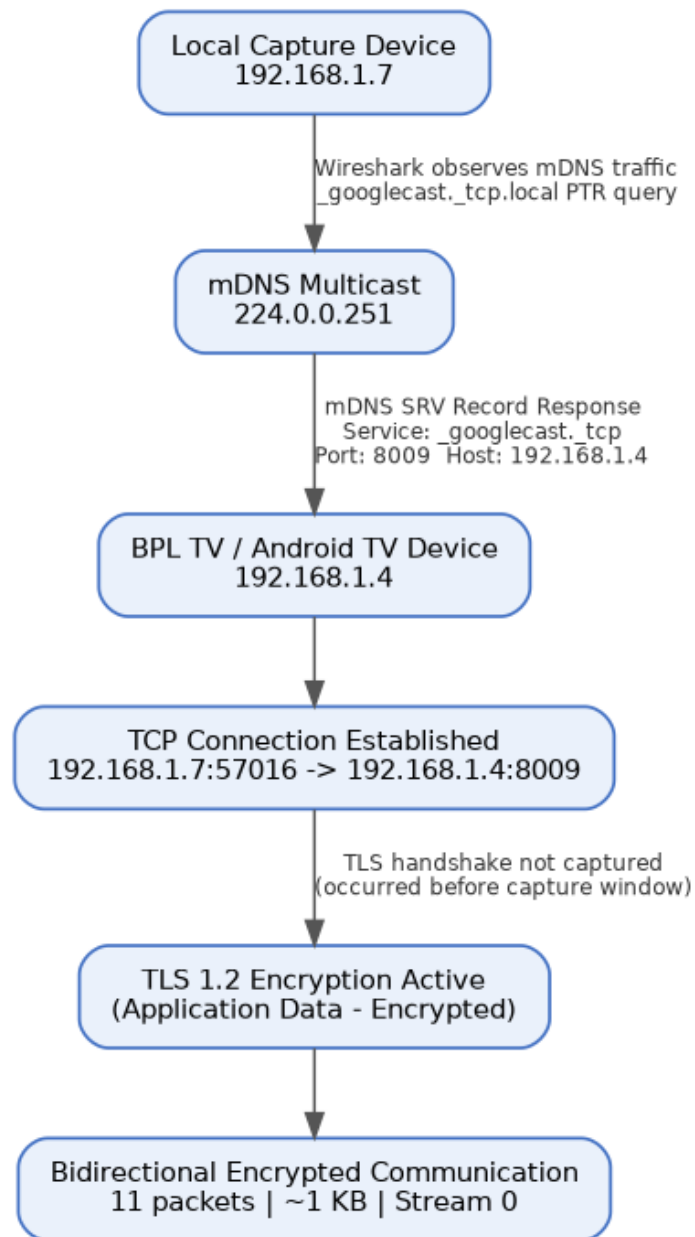
The following table summarizes each protocol observed in this investigation:

Protocol	What it does	How it appeared in this investigation
IPv4	The primary Internet Protocol used to address and route packets between devices using 32-bit addresses.	Most observed traffic used IPv4 addressing. Devices such as 192.168.1.7 and 192.168.1.4 communicated over IPv4.
IPv6	A newer Internet Protocol using 128-bit addresses, often used alongside IPv4.	IPv6 packets were observed, including link-local addresses used in mDNS traffic (e.g., fe80::e62:a6ff:fea6... and ff02::fb).
TCP	Transmission Control Protocol. A reliable, connection-oriented transport protocol that ensures data arrives in order.	TCP was used for the connection between 192.168.1.7 (port 57016) and 192.168.1.4 (port 8009), carrying TLS-encrypted data.
UDP	User Datagram Protocol. A faster, connectionless transport protocol used where speed matters more than reliability.	mDNS and DNS traffic both used UDP as their transport.
TLS (v1.2)	Transport Layer Security. A cryptographic protocol that encrypts data so it cannot be read by third parties.	TLS 1.2 was observed encrypting the application data on the 192.168.1.7 ↔ 192.168.1.4:8009 connection and on 192.168.1.7 → 4.150.223.96:443.
mDNS	Multicast DNS. A protocol used on local networks to discover devices and services without a central DNS server.	Extensively observed. 192.168.1.4 broadcast mDNS queries and responses advertising itself as "BPL TV" with Google Cast and Android TV services.
DNS	Domain Name System. Translates human-readable domain names into IP addresses.	A DNS query for mobile.events.data.microsoft.com was captured, with the response resolving it to 4.150.223.96 via Azure.
QUIC	A modern transport protocol built on UDP, used by Google services and HTTP/3 for faster, encrypted communication.	QUIC traffic was observed in the protocol hierarchy, indicating Google service communication in the background.
ARP	Address Resolution Protocol. Maps an IP address to a physical (MAC) address on a local network.	ARP packets were observed in the protocol hierarchy, used for local network address resolution.

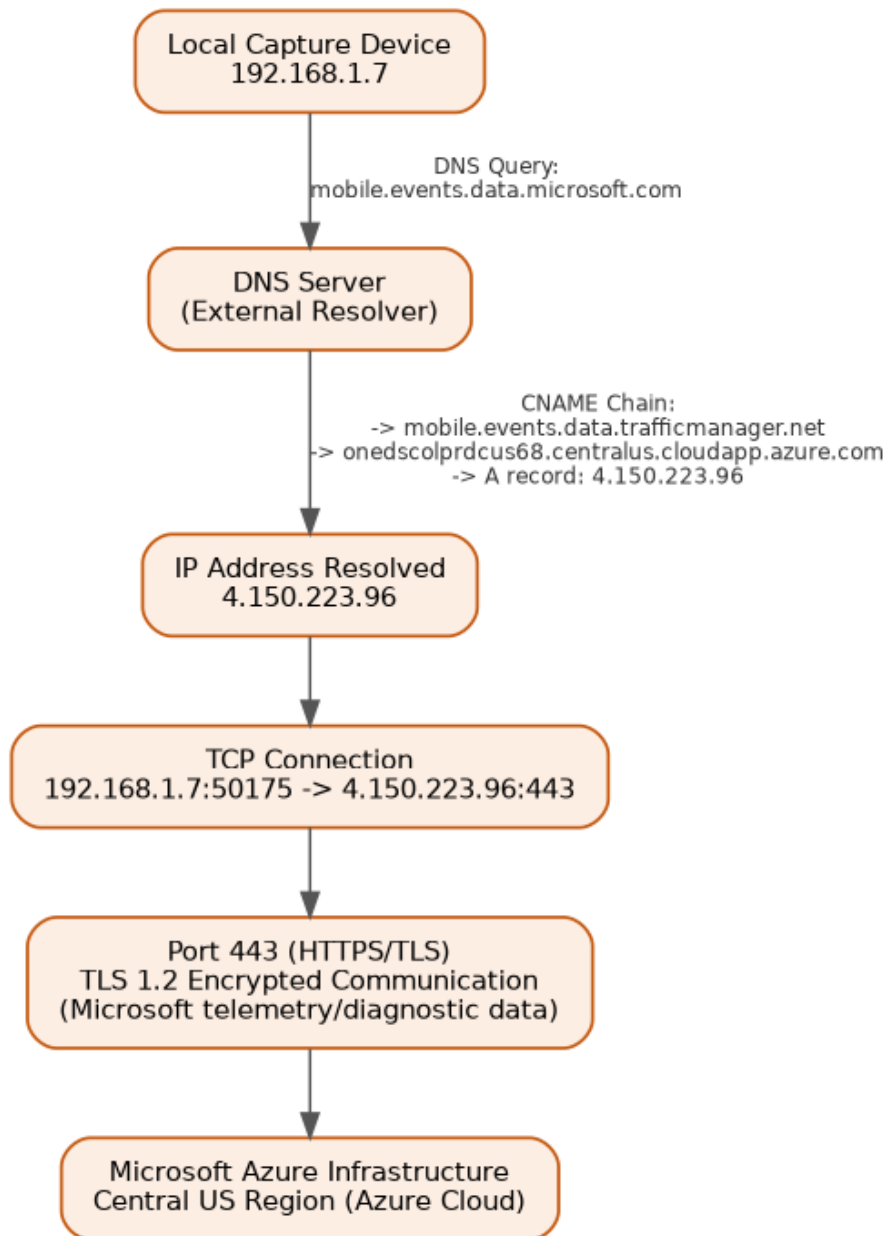
9. Traffic Flow Diagram

The following diagrams represent the two main communication flows identified in this investigation.

Flow A - Google Cast Service Discovery and Connection



Flow B - DNS Resolution and HTTPS Connection to Microsoft



10. Key Findings

The following are the most important evidence-based conclusions from this investigation:

- Device 192.168.1.4 was positively identified as a BPL Android TV based on mDNS service name advertisements ("BPL TV._androidtvremote2._tcp.local") captured in the packet traffic.
- 192.168.1.4 advertises a Google Cast service (_googlecast._tcp.local) via mDNS, consistent with a Chromecast-capable smart TV.
- A captured DNS SRV record directly and explicitly linked the Google Cast service to TCP port 8009 on 192.168.1.4 - the port-service relationship was established from captured packet evidence, not assumed.
- An active TCP connection was observed between 192.168.1.7 (port 57016) and 192.168.1.4 (port 8009), comprising 11 packets and approximately 1 KB of data.
- All application data on the 192.168.1.7 ↔ 192.168.1.4:8009 connection was encrypted using TLS 1.2, making the payload unreadable from the packet capture.
- The TLS handshake for the Google Cast connection was not captured - the connection was already established when the capture began. This is a limitation of the capture window, not evidence of abnormal behavior.
- A DNS query for mobile.events.data.microsoft.com was captured and resolved through a CNAME chain to 4.150.223.96 (Microsoft Azure, Central US).
- A TCP connection from 192.168.1.7 (port 50175) to 4.150.223.96 (port 443) was found in the capture, demonstrating the complete DNS resolution → TCP connection workflow.
- An additional TCP connection from 172.217.70.188 (source port 5228) to 192.168.1.7 (port 60971) was observed, consistent with Google push notification (FCM) infrastructure and not indicative of suspicious activity.
- The protocol hierarchy revealed 132 total frames dominated by QUIC (22.6% of bytes), TLS, mDNS, and DNS - typical of a modern Wi-Fi network with encrypted internet traffic and smart devices.

11. Security Analysis

This investigation offers several important insights from a defensive cybersecurity perspective.

Value of Packet Capture

Packet capture with Wireshark allows a network analyst to observe exactly what is happening on a network at a low level. Even without application-level access, a defender can identify which devices are present, what services they are running, which IP addresses they are communicating with, and how much data is being exchanged. This visibility is valuable for network inventory, anomaly detection, and incident investigation.

Protocol Identification

Understanding the protocols present in a capture is a prerequisite for any deeper analysis. Knowing that mDNS is responsible for local service discovery prevents misidentifying those broadcasts as malicious. Knowing that QUIC represents encrypted Google traffic prevents it from appearing suspicious. Protocol knowledge is the foundation of network analysis.

The Impact of Encryption

A significant portion of this capture was encrypted - TLS 1.2 on the Google Cast connection, TLS 1.2 on the Microsoft Azure connection, and QUIC (which is always encrypted) for Google services. Encryption protects users' privacy and prevents eavesdropping, but it also means that a network analyst cannot read application-layer content from a packet capture. Defenders who rely on Deep Packet Inspection (DPI) are increasingly limited by widespread encryption. For analysts, this reinforces the importance of metadata analysis: even without reading the payload, IP addresses, ports, timing, and volume of traffic provide useful signals.

mDNS and DNS as Investigative Tools

While encryption hides payload content, DNS and mDNS traffic often remain visible and informative. DNS queries reveal which domain names a device is resolving, which can indicate what services it is communicating with. mDNS advertisements reveal what services local devices are running. In this investigation, mDNS was the primary tool for identifying the BPL TV device and the Google Cast service - without it, 192.168.1.4 would have appeared only as an unknown IP address with encrypted traffic.

Port Numbers Require Context

Port numbers should never be interpreted in isolation. Port 8009 is not a universally known port - without the SRV record from mDNS, its association with Google Cast would have required additional research. Port 5228 (used by Google FCM) is another example of a non-standard port that is completely legitimate. Treating unfamiliar ports as inherently suspicious without contextual evidence leads to poor analysis and false positives.

Correlation is Key

The most valuable findings in this investigation came from correlating multiple pieces of evidence: the mDNS SRV record linked a service name to a port number; the DNS response linked a domain name to an IP address; the TCP packet confirmed an actual connection to that IP address. No single packet told the full story - the narrative emerged from combining evidence across multiple packets and protocols.

Capture Limitations

The absence of the TLS handshake in this capture is a reminder that packet captures are snapshots in time. Events that occurred before the capture started, on other network interfaces, or in encrypted channels may not be visible. A clean capture does not necessarily mean nothing happened - and the presence of certain traffic does not guarantee the absence of other threats.

No Evidence of Malicious Activity

It is important to state clearly: nothing in this capture constitutes evidence of malicious activity. The Google Cast traffic between the local machine and the BPL TV is normal smart TV behavior. The Microsoft telemetry traffic is normal Windows/application telemetry. The Google FCM connection is normal push notification infrastructure. A cybersecurity investigation must always be evidence-based - it is equally important to conclude that traffic is benign when the evidence supports that conclusion.

12. Limitations

- The capture represents only a limited, specific period of network activity. Traffic that occurred before or after the capture window is not reflected.
- The TLS handshake for the 192.168.1.7 ↔ 192.168.1.4:8009 connection was not captured because the connection was already established when Wireshark began recording. No conclusions about the handshake parameters (cipher suite, certificate, etc.) can be drawn from this capture.
- All application-layer payload data within TLS-encrypted connections is unreadable without the server's private key. The content of communications with 192.168.1.4:8009, 4.150.223.96:443, and QUIC connections cannot be determined from this capture.
- The DNS response for mobile.events.data.microsoft.com proves name resolution occurred but does not, by itself, prove that the application successfully completed a connection, sent data, or received a meaningful response from the server.
- The protocol hierarchy shows 132 frames, which is a small sample of real-world network traffic. A longer capture would provide a more representative picture of normal network behavior for this network.
- Some packets in observed TCP conversations are marked as "[Conversation completeness: Incomplete]", indicating that not all packets of those sessions were captured. This is normal when a capture starts mid-connection.
- The investigator did not have access to the private keys or application logs of any device, limiting analysis to what could be observed in the network traffic alone.

13. Conclusion

This project successfully demonstrated a structured, evidence-based approach to network traffic analysis using Wireshark. Starting from a simple packet capture on a home Wi-Fi network, the investigation progressively identified devices, discovered services, established port-service relationships from captured evidence, examined encryption, and correlated DNS resolutions with TCP connections.

The investigation showed that 192.168.1.4 - identified as a BPL Android TV - was advertising a Google Cast service via mDNS, with the service confirmed by a captured SRV record to be running on TCP port 8009. The TCP connection between the local machine and this service was observed and analyzed, including confirmation of TLS 1.2 encryption that prevented direct payload inspection. The protocol hierarchy provided context for the overall capture, revealing a mix of protocols typical of a modern Wi-Fi network.

The DNS investigation demonstrated how a domain name query (mobile.events.data.microsoft.com) resolves through CNAME chains to a final IP address (4.150.223.96) and how that resolution can be correlated with an observed TCP connection to port 443 - illustrating the complete DNS-to-connection communication workflow.

Perhaps most importantly, this project demonstrated the discipline of evidence-based analysis: drawing conclusions only from what the packets show, acknowledging what cannot be confirmed from the capture, and avoiding speculation or assumption. The absence of a TLS handshake in the capture was not treated as evidence that no handshake occurred. An unfamiliar port number was not treated as suspicious without corroborating context. A DNS response was not treated as proof of a completed connection without the corresponding TCP traffic.

Wireshark is a powerful tool, but its power lies in the analyst's ability to read, interpret, and reason carefully about the evidence it presents. This project provided a practical, hands-on foundation for developing that skill.

14. Skills Learned

Through this investigation, the following practical network analysis skills were developed and demonstrated:

- Packet capture - Starting a live capture on a Wi-Fi interface and allowing Wireshark to record real network traffic.
- Wireshark display filtering - Using filters such as `ip.addr ==`, `mdns`, `tcp.stream eq`, `tls.handshake`, and combined filters using `&&` to isolate specific traffic.
- Reading packet headers - Interpreting source/destination IP addresses, MAC addresses, TCP port numbers, sequence numbers, and acknowledgment numbers.
- IPv4 and IPv6 analysis - Understanding the difference between IPv4 and IPv6 addressing and recognizing both in a capture.
- TCP analysis - Understanding TCP streams, stream indices, bidirectional communication, ACK packets, and retransmissions.
- Port analysis - Understanding ephemeral ports vs. well-known ports, and interpreting port-service relationships from contextual evidence.
- DNS analysis - Interpreting DNS A records, CNAME records, and the CNAME chain resolution process.
- mDNS and service discovery - Understanding how mDNS enables zero-configuration service discovery and how to interpret PTR, SRV, TXT, and A records from mDNS responses.
- TLS analysis - Recognizing TLS-encrypted traffic, understanding why payload inspection fails, and identifying TLS version from packet details.
- Following TCP streams - Using the "Follow TCP Stream" feature to reconstruct application-layer conversations.
- Protocol hierarchy analysis - Using the Protocol Hierarchy Statistics window to gain an overview of a capture's composition.
- Conversation statistics - Using the Conversations window to identify active TCP connections and summarize packet counts and data volumes.
- Evidence-based traffic interpretation - Distinguishing between what a capture proves and what it does not prove, and drawing defensible conclusions from captured evidence.

15. Future Improvements

The following are realistic next steps to build on the skills demonstrated in this project:

- Capture a complete TLS handshake - Start Wireshark before initiating a new connection to capture the ClientHello, ServerHello, Certificate, and key exchange messages, and examine the cipher suite and certificate details.
- Compare HTTP and HTTPS - Capture traffic from a site that still supports plain HTTP and compare the readable payload with the encrypted output of an HTTPS site, demonstrating encryption's effect directly.
- Investigate ARP - Perform a dedicated ARP analysis, examining how devices map IP addresses to MAC addresses and understanding what an ARP spoofing attack would look like in a capture.
- Investigate DHCP - Capture the DHCP four-way handshake (Discover, Offer, Request, Acknowledge) to understand how IP addresses are assigned dynamically on a network.
- Analyze DNS more deeply - Investigate DNS response times, caching behavior (TTL values), and examine what happens when a DNS query fails or times out.
- Compare TCP and UDP - Capture both TCP-based (e.g., HTTPS) and UDP-based (e.g., DNS, QUIC) traffic side by side to understand the difference in reliability and overhead.
- Perform a larger traffic capture - Capture traffic over a longer period (30 minutes to an hour) and use the Protocol Hierarchy and Conversations panels to identify the most active connections and protocols.
- Investigate suspicious traffic in a controlled lab environment - Set up a virtual machine running a deliberately vulnerable application (using platforms like TryHackMe or HackTheBox) and practice identifying malicious traffic patterns in a safe, legal environment.
- Learn to use Wireshark's IO Graphs - Use the Statistics → I/O Graphs feature to visualize traffic volume over time and identify traffic spikes or unusual patterns.
- Explore Wireshark coloring rules - Understand and customize Wireshark's color coding to make specific types of traffic easier to identify at a glance.

Overall Investigation Summary

This investigation captured 132 network frames on a Wi-Fi interface and systematically examined them to understand what devices were present, what services they were running, and how they were communicating.

The starting point was filtering traffic by the IP address 192.168.1.4, which revealed mDNS service advertisements identifying the device as a BPL Android TV. Drilling into the mDNS traffic, a specific Google Cast service advertisement was found. A captured mDNS SRV record directly stated that Google Cast was running on TCP port 8009 at 192.168.1.4 - establishing the port-service relationship from captured evidence.

A TCP connection between the local machine (192.168.1.7, port 57016) and the BPL TV (192.168.1.4, port 8009) was examined in detail. The connection carried TLS 1.2-encrypted application data. The Follow TCP Stream output was unreadable - confirming encryption was working as intended. A search for the TLS handshake returned no results, because the connection predated the capture window. The TLS version (1.2) was confirmed from the Application Data packet headers.

The protocol hierarchy provided context: 132 frames containing IPv4, IPv6, TCP, UDP, TLS, QUIC, mDNS, DNS, and ARP - a normal modern Wi-Fi traffic mix.

A DNS query for mobile.events.data.microsoft.com was found and traced through a CNAME chain to 4.150.223.96. A TCP connection to that IP on port 443 was confirmed, demonstrating the complete DNS resolution → TCP connection → TLS encryption workflow. A Google push notification connection (172.217.70.188:5228 → 192.168.1.7:60971) was also documented and identified as legitimate background infrastructure traffic.

Throughout the investigation, conclusions were limited to what the evidence directly supported. No malicious activity was observed. The investigation demonstrated that careful, methodical packet analysis - even at a beginner level - can reveal meaningful, accurate information about network behavior, device identity, service operation, and communication patterns.